# memory_layers_agent — 이중 계층 메모리(Session 단기 vs Memory 장기) 에이전트

`memory_layers_agent` 는 Google Agent Development Kit (ADK)의 **이중 계층 메모리(Dual-Layer Memory)** 아키텍처를 실습하고 검증하기 위한 지능형 에이전트입니다. 대화의 임시 상태와 활성 스레드를 다루는 **Session Service(단기 메모리)**와, 세션을 넘어 영속적인 사실과 사용자 지식을 유지·인출하는 **Memory Service(장기 메모리)**를 결합하여 사용자 맞춤형 연속 대화를 수행합니다.

> **관련 강의노트**: Part 2 · 02. Google ADK 아키텍처 (슬라이드 29: 이중 계층 메모리)

---

## 1. 이중 계층 메모리 비교 매트릭스

| 비교 차원 | Session Service (단기 메모리) | Memory Service (장기 메모리) |
|---|---|---|
| **저장 범위** | 활성 대화 스레드 및 세션 임시 상태 | 세션을 넘어서는 영속적인 사용자 사실·선호·지식 |
| **저장 데이터** | 메시지 이력, 도구 호출 결과, State 변수 | 구조화/추출된 사용자 프로필, 과거 대화 사건 및 선호 정보 |
| **생명주기** | 대화 스레드 종료 시 소멸 (단, DB 세션 사용 시 세션 키로 보존) | 세션 간 영구 보존 |
| **구현체** | `InMemorySessionService` , `DatabaseSessionService` , `VertexAiSessionService` | `InMemoryMemoryService` , `VertexAiMemoryBankService` |
| **에이전트 접근 방식** | 이벤트 이력 자동 주입, `{key}` 지시문 템플릿, `tool_context.state` | `preload_memory` (매 턴 자동 사전 검색), `load_memory` (에이전트 능동 검색 도구) |

---

## 2. 세션 State 변수 접두사(Prefix) 규칙

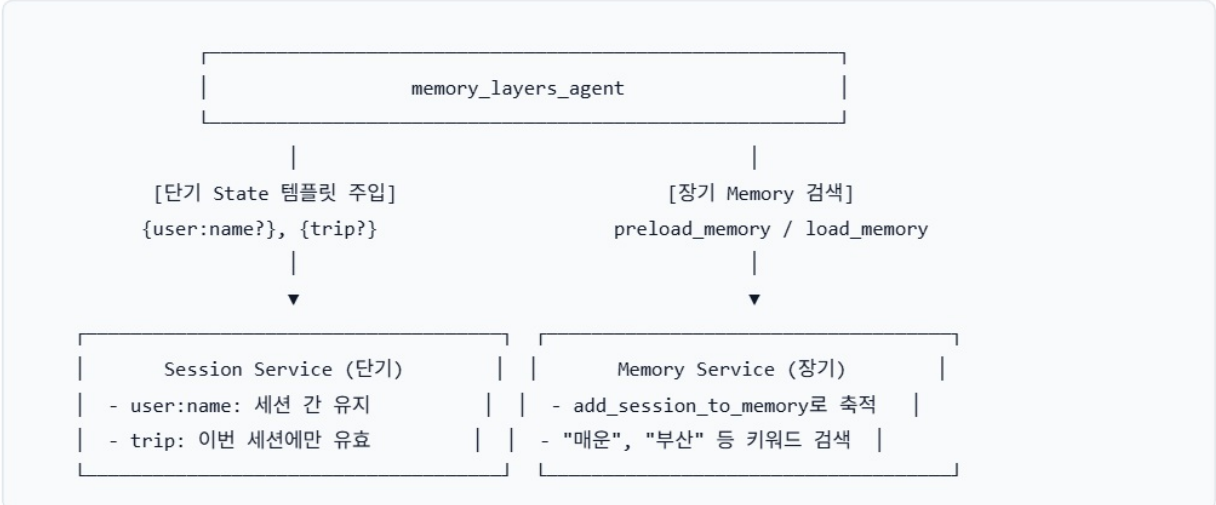
ADK의 Session State는 키의 접두사(Prefix)에 따라 저장 범위와 공유 스코프가 엄격하게 결정됩니다:

- `user:<key>` (예: `user:name`):
  - **사용자 스코프**: 동일한 사용자( `user_id` )가 생성한 **모든 대화 세션 간에 자동 공유 및 유지**됩니다.
  - 사용자 이름, 직책, 기본 설정 등 영속적 프로필 저장에 적합합니다.
- **접두사 없음** (예: `trip` ):
  - **세션 스코프**: 해당 대화 세션( `session_id` ) 내부에서만 유효합니다. 새 세션이 시작되면 완전히 초기화됩니다.
  - 일회성 여행 계획, 현재 처리 중인 주문 번호 등 임시 컨텍스트에 적합합니다.
- `app:<key>` (예: `app:notice` ):
  - **애플리케이션 전역 스코프**: 애플리케이션( `app_name` ) 내의 모든 사용자와 모든 세션이 공유하는 전역 설정값입니다.
- `temp:<key>` (예: `temp:raw` ):
  - **일회성 호출 스코프**: 단일 호출 턴(Turn) 동안만 메모리에 임시 유지되며, 세션 저장소(DB 등)에 영구 기록되지 않습니다.

---

## 3. 에이전트 동작 원리 및 도구 구성

`memory_layers_agent` 는 지시문(Instruction) 템플릿 치환과 전용 도구를 통해 두 메모리 계층을 모두 활용합니다:



## ① 지시문(Instruction) 템플릿

```
instruction="""
너는 개인 비서다.
[단기: 이번 세션 State]
  사용자 이름: {user:name?}
  이번 세션의 계획: {trip?}
규칙
1) 사용자가 이름을 말하면 save_profile, 출장·여행 계획을 말하면 save_trip 을 호출한다.
2) 이번 세션 State 에 없는 과거 정보(음식 취향, 지난 대화의 계획 등)는
   자동으로 제공되는 과거 기억을 먼저 보고, 부족하면 load_memory 로 검색한다.
   load_memory 검색어는 조사를 붙이지 않은 핵심 단어 하나씩 쓴다 (예: "매운", "부산", "출장").
3) 기억에도 없으면 모른다고 말한다. 지어내지 않는다.
4) 답은 2~3문장으로 한다.
"""
```

- `{user:name?}` : 세션 State에 `user:name` 이 존재하면 값을 주입하고, 없으면 물음표( `?` ) 문법에 따라 빈 값으로 안전하게 대체됩니다.
- `{trip?}` : 이번 세션에 등록된 계획 정보를 프롬프트에 자동으로 주입합니다.

## ② 도구 구성 (Tools)

1. `save_profile(name, tool_context)` :
   - `tool_context.state["user:name"] = name`
   - 사용자 이름을 영속적인 `user:` 스코프 State로 저장합니다. 다른 세션에서도 즉시 공유됩니다.
2. `save_trip(trip, tool_context)` :
   - `tool_context.state["trip"] = trip`
   - 출장이나 여행 일정을 현재 세션 스코프 State로 저장합니다.
3. `preload_memory` :
   - Google ADK 내장 도구. 매 턴마다 사용자의 입력 문장에 포함된 키워드를 기반으로 Memory Service를 자동 사전 검색하여 프롬프트 컨텍스트에 주입합니다.
4. `load_memory` :
   - Google ADK 내장 도구. 모델이 추론 중 추가적인 과거 기억(음식 취향, 과거 이력 등)이 필요하다고 판단할 때 능동적으로 장기 기억을 검색합니다.

---

## 4. 파일 구성

```
memory_layers_agent/
├── __init__.py          # agent 모듈 및 root_agent 객체 노출
├── agent.py             # save_profile, save_trip 도구, 지시문 및 root_agent 정의
├── README-KR.md         # 한국어 상세 매뉴얼 (본 문서)
└── README-EN.md         # 영문 번역 매뉴얼
```

- `agent.py` :
  - `save_profile(name: str, tool_context: ToolContext) -> dict` : 사용자 이름 저장 ( `user:name` )
  - `save_trip(trip: str, tool_context: ToolContext) -> dict` : 현재 세션 일정 저장 ( `trip` )
  - `preload_memory` , `load_memory` : ADK 공식 메모리 도구 등록
  - `root_agent` : Gemini 기반 에이전트 인스턴스

---

## 5. 사전 환경 준비 및 설정

### 1) 의존성 패키지 설치

```
pip install google-adk python-dotenv
# 또는 uv 사용 시
uv pip install google-adk python-dotenv
```

### 2) 환경 변수( `.env` ) 설정

상위 디렉터리( `part02_architecture` 또는 프로젝트 루트)에 `.env` 파일을 생성하고 설정합니다:

```
# Gemini Developer API 키
GEMINI_API_KEY=your_gemini_api_key_here

# 또는 Vertex AI 사용 시
# GOOGLE_GENAI_USE_VERTEXAI=TRUE
# GOOGLE_CLOUD_PROJECT=your-gcp-project-id
# GOOGLE_CLOUD_LOCATION=us-central1

# 에이전트 모델 설정
ADK_MODEL=gemini-flash-latest
```

---

## 6. 수행 및 실행 방법

### 방법 1: 터미널 대화형 인터랙티브 모드 ( `adk run` )

터미널에서 직접 에이전트와 대화하며 세션 State 저장 과정을 확인합니다:

```
# adk-based_Agent 폴더에서 실행
adk run memory_layers_agent
```

또는 상위 폴더 `part02_architecture` 에서 실행:

```
cd ..
adk run adk-based_Agent/memory_layers_agent
```

실행 후 다음과 같이 대화하며 동작을 확인합니다:

```
User: 안녕, 내 이름은 홍길동이고 다음 주 월요일 제주도 출장이 있어.
Agent: 홍길동님 반갑습니다. 다음 주 월요일 제주도 출장 일정을 기억하겠습니다.
```

### 방법 2: 웹 개발 UI 환경 ( `adk web` )

Google ADK 브라우저 GUI를 통해 대화를 진행하고, 세션 State 변수와 이벤트 로그를 실시간으로 확인합니다:

```
# adk-based_Agent 폴더에서 실행
adk web --port 8008
```

브라우저로 `http://localhost:8008` 에 접속한 뒤 좌측 상단에서 ** `memory_layers_agent` **를 선택합니다. 대화 중 도구 호출 파라미터와 `state_delta` 가 어떻게 누적되는지 시각적으로 관찰할 수 있습니다.

### 방법 3: 메모리 계층 전용 검증 스크립트 실행 ( `run_memory_layers.py` )

상위 `Direct_ADK` 폴더의 검증 스크립트를 통해 단기 메모리(Session)와 장기 메모리(Memory)의 저장 범위, 생명주기, 인출 메커니즘을 단계별로 검증합니다:

```
# part02_architecture 폴더에서 실행

# 1) 오프라인 모드: LLM 호출 없이 Service API 레벨에서 저장 구조와 생명주기 직접 확인
uv run Direct_ADK/run_memory_layers.py offline

# 2) LLM 연동 모드: Gemini 모델을 통해 세션 1 저장 ➡️ 세션 2 장기 기억 인출 검증
uv run Direct_ADK/run_memory_layers.py llm

# 3) 전체 실행 (기본값)
uv run Direct_ADK/run_memory_layers.py all
```

### 방법 4: 파이썬 코드를 통한 멀티 세션 기억 유지 검증

두 개의 연속된 세션을 생성하여 장기 메모리 전이 과정을 확인하는 코드 예시입니다:

```
import asyncio
from google.adk.sessions import InMemorySessionService
from google.adk.memory import InMemoryMemoryService
from common.runner_util import Session
from memory_layers_agent.agent import root_agent

async def main():
    session_service = InMemorySessionService()
    memory_service = InMemoryMemoryService()
    app, user = "memory_demo", "user_kim"

    # [세션 1] 개인 정보 및 취향 저장
    print("=== 세션 1 시작 ===")
    s1 = Session(root_agent, app_name=app, user_id=user,
                 session_service=session_service, memory_service=memory_service)
    await s1.send("안녕! 나는 김철수야. 매운 음식을 전혀 못 먹어. 다음 주 화요일에 부산 출장이 있어.")

    # 세션 종료 후 장기 기억으로 이관
    current_s1 = await s1.get_session()
    await memory_service.add_session_to_memory(current_s1)
    print("세션 1 State:", current_s1.state)

    # [세션 2] 새 세션 시작 (새로운 세션 ID)
    print("\n=== 세션 2 시작 (새 대화 스레드) ===")
    s2 = Session(root_agent, app_name=app, user_id=user,
                 session_service=session_service, memory_service=memory_service)
    # preload_memory 검색을 위해 '부산' 키워드 포함 질의
    res = await s2.send("내 이름이 뭐였지? 그리고 이번 부산 출장 때 먹을 저녁 메뉴 하나만 추천해줘.")
    print("에이전트 답변:", res)
    print("세션 2 State:", (await s2.get_session()).state)

if __name__ == "__main__":
    asyncio.run(main())
```

---

## 7. 시나리오 동작 예시

## [세션 1] 프로필과 단기 계획 및 취향 전달

- **사용자**: "안녕, 나는 김철수야. 매운 음식을 못 먹고, 다음 주 화요일에 부산 출장이 있어."
- **에이전트 동작**:
  1. `save_profile(name="김철수")` 호출 → `state["user:name"] = "김철수"` 저장 (사용자 전역 스코프)
  2. `state["trip"] = "다음 주 화요일 부산 출장"`
  3. 세션 종료 시 `memory_service.add_session_to_memory(session)`를 통해 세션 전체 대화가 장기 기억(Memory Service)으로 이관

## [세션 2] 새 대화 시작 (새 세션 ID)

- **상태 비교**:
  - `trip` 변수는 이전 세션의 종료와 함께 초기화되어 세션 2에는 존재하지 않음.
  - `user:name`은 사용자 스코프(`user:`)이므로 새 세션 State에도 그대로 유지됨.
- **사용자**: "내 이름이 뭐였지? 그리고 이번 부산 출장 때 먹을 저녁 메뉴 하나만 추천해줘."
- **에이전트 동작**:
  1. 지시문 템플릿 `{user:name?}`을 통해 사용자 이름 "김철수"를 세션 State에서 즉각 인지.
  2. 질문의 '부산' 키워드를 바탕으로 `preload_memory` 및 `load_memory`가 장기 기억에서 "매운 음식을 못 먹는다"는 과거 선호 정보를 검색·인출.
- **최종 응답**:

> "김철수님, 매운 음식을 피하셔야 하므로 자극적이지 않고 담백한 부산 돼지국밥이나 신선한 생선구이를 추천해 드립니다."

---

## 8. 문제 해결 (FAQ & Troubleshooting)

| 현상 | 원인 | 해결 방법 |
| --- | --- | --- |
| 새 세션에서 사용자 이름이 기억나지 않음 | `user:` 접두사 없이 State에 저장되었거나 세션 서비스 인스턴스가 재생성됨 | `save_profile` 도구가 `tool_context.state["user:name"]` 형태로 키를 저장하는지 확인합니다. |
| 새 세션에서 과거 취향(예: 매운 음식 불가)을 찾지 못함 | 세션 1 종료 시 `memory_service.add_session_to_memory(s1)`가 호출되지 않음 | 대화 세션 종료 후 장기 메모리 저장 메서드가 정상 호출되었는지 확인합니다. |
| `InMemoryMemoryService` 검색 결과가 비어 있음 | `InMemory` 구현체는 키워드의 완전 일치(Exact substring) 방식을 사용함 | 사용자의 질문에 장기 기억에 저장된 핵심 단어(예: "부산", "매운")가 포함되어 있는지 확인합니다. 프로덕션 환경에서는 시맨틱 벡터 검색을 지원하는 `VertexAiMemoryBankService`를 사용합니다. |